PRÁCTICA 1 – ORGANIZACIÓN DE FICHEROS (ASF)
Administración de Bases de Datos

Francisco Javier Santos Rivas

26 de marzo de 2026

Índice

1. Introducción **2**

2. TDA `fileRecord` **2**
2.1. Interfaz del TDA (`fileRecord.h`) 2
2.2. Representación en memoria 3
2.3. Operaciones públicas 3

3. TDA `fileASF` **4**
3.1. Interfaz del TDA (`fileASF.h`) 4
3.2. Representación en memoria 5
3.2.1. Contenido de `fileHeader` 6
3.2.2. Contenido de `blockInMemory` 6
3.3. Operaciones públicas 6
3.4. Operaciones privadas 8

4. Análisis de operaciones **10**
4.1. Operaciones que realizan lectura o escritura en disco 10
4.2. Operaciones que escriben `fileHeader` en disco 12
4.3. Operaciones que abren o cierran `diskFile` 12
4.4. Operaciones que desplazan la posición actual del fichero 12

1. Introducción

La práctica consiste en diseñar los tipos de dato abstractos necesarios para gestionar ficheros con organización ASF (Archivo Secuencial Físico) que almacenan registros de longitud fija en bloques homogéneos.

El diseño se centra en dos TDA:

- `fileASF`, encargado de representar y gestionar el fichero ASF en memoria.
- `fileRecord`, encargado de representar un registro intercambiado con el fichero.

En ambos casos se sigue el criterio indicado en el enunciado: tanto la cabecera del fichero como el bloque actual en memoria y el propio registro se representan como secuencias consecutivas de bytes, sin recodificaciones intermedias.

2. TDA `fileRecord`

2.1. Interfaz del TDA (`fileRecord.h`)

```
#ifndef FILERECORD_H
#define FILERECORD_H

#include <vector>

enum campType : int {
    LOGICO = 0,
    CARACTER = 1,
    ENTERO = 2,
    REAL = 3,
    CADENA = 4
};

class fileRecord {
private:
    std::vector<char> recordStructure;
    std::vector<char> recordData;

public:
    fileRecord();
    fileRecord(const std::vector<char>& structure,
               const std::vector<char>& data);

    void setRecordStructure(const std::vector<char>& structure);
    void setRecordData(const std::vector<char>& data);

    const std::vector<char>& getRecordStructure() const;
    const std::vector<char>& getRecordData() const;
};

#endif
```

2.2. Representación en memoria

La representación del TDA `fileRecord` está formada por dos secuencias de bytes:

- `recordStructure` (`std::vector<char>`): secuencia de bytes que almacena la estructura del registro.
- `recordData` (`std::vector<char>`): secuencia de bytes que almacena los valores de los campos del registro.

El contenido de `recordStructure` es, en este orden:

- un `int` con el tamaño de la cabecera del registro,
- un `int` con el número de atributos del registro,
- una secuencia de `int` con el tipo de cada campo,
- una secuencia de `int` con la longitud de cada campo.

El contenido de `recordData` es una secuencia de bytes con los valores de todos los campos del registro, almacenados en el mismo orden definido por `recordStructure`.

El TDA `fileRecord` no incluye ni el identificador del registro dentro del fichero ni el estado del registro. Ambos datos pertenecen exclusivamente al registro almacenado dentro de un bloque del fichero ASF.

2.3. Operaciones públicas

- `fileRecord()`
  Constructor por defecto. Crea un registro sin estructura ni datos.
- `fileRecord(`
  `const std::vector<char>& structure,`
  `const std::vector<char>& data)`
  Constructor parametrizado. Crea un registro a partir de la secuencia de bytes que representa su estructura y de la secuencia de bytes que representa sus datos.
- `void setRecordStructure(`
  `const std::vector<char>& structure)`
  Sustituye la estructura del registro por la secuencia de bytes recibida.
- `void setRecordData(`
  `const std::vector<char>& data)`
  Sustituye los datos del registro por la secuencia de bytes recibida.
- `const std::vector<char>& getRecordStructure() const`
  Devuelve la secuencia de bytes que representa la estructura del registro.
- `const std::vector<char>& getRecordData() const`
  Devuelve la secuencia de bytes que representa los datos del registro.

3. TDA `fileASF`

3.1. Interfaz del TDA (`fileASF.h`)

```
#ifndef FILEASF_H
#define FILEASF_H

#include <string>
#include <vector>
#include <fstream>
#include "fileRecord.h"

enum recordState : int {
    BORRADO = 0,
    ACTIVO = 1
};

class fileASF {
private:
    std::string diskFilename;
    std::fstream diskFile;
    std::vector<char> fileHeader;
    std::vector<char> blockInMemory;
    bool hasHeaderChanged;
    bool hasBlockChanged;
    int loadedBlockId;
    int currentBlockId;
    int currentRecordIndex;

    int getHeaderSize();
    int getBlockCount();
    int getRecordCount();
    int getBlockSize();
    int getBlockHeaderSize();
    int getRecordHeaderSize();
    int getAttributeCount();
    int getRecordDataSize();
    int getRecordSize();
    int getRecordsPerBlock();

    std::vector<char> buildRecordStructure();

    void readBlockFromDisk(int blockId);
    void writeBlockToDisk();
    void readFileHeaderFromDisk();
    void writeFileHeaderToDisk();

    int readInt(const std::vector<char>& buffer, int offset);
    void writeInt(std::vector<char>& buffer, int offset, int value);

public:
```

```
    fileASF();

    void create(const std::string& filename,
                int blockSize,
                int blockHeaderSize,
                int recordHeaderSize,
                const std::vector<campType>& types,
                const std::vector<int>& lengths);

    void open(const std::string& filename);
    void close();

    void insertRecord(const fileRecord& record);
    bool readNextRecord(fileRecord& record);
    bool findRecord(int recordId, fileRecord& record);
    bool deleteRecord(int recordId);
    bool modifyRecord(int recordId, const fileRecord& record);
    void resetRead();
};

#endif
```

3.2. Representación en memoria

El TDA `fileASF` mantiene en memoria toda la información necesaria para gestionar un fichero con organización ASF.

- `diskFilename` (`std::string`): nombre y ruta del fichero en disco.
- `diskFile` (`std::fstream`): estructura de datos que permite acceder al fichero almacenado en disco.
- `fileHeader` (`std::vector<char>`): secuencia de bytes que almacena la cabecera del fichero exactamente con el mismo formato con el que se lee o se escribe en disco.
- `blockInMemory` (`std::vector<char>`): secuencia de bytes que almacena el bloque actualmente cargado en memoria, también con el mismo formato con el que se lee o se escribe en disco.
- `hasHeaderChanged` (`bool`): centinela que indica si la cabecera del fichero ha sido modificada desde la última escritura en disco.
- `hasBlockChanged` (`bool`): centinela que indica si el bloque en memoria ha sido modificado desde la última escritura en disco.
- `loadedBlockId` (`int`): identificador del bloque actualmente cargado en `blockInMemory`.
- `currentBlockId` (`int`): identificador del bloque correspondiente a la posición actual de lectura secuencial.
- `currentRecordIndex` (`int`): índice del registro dentro del bloque actual correspondiente a la posición actual de lectura secuencial.

3.2.1. Contenido de `fileHeader`

La secuencia de bytes almacenada en `fileHeader` contiene, en este orden:

- un `int` en el offset 0 con el tamaño de la cabecera del fichero,
- un `int` en el offset 4 con el número de bloques del fichero,
- un `int` en el offset 8 con el número total de registros insertados en el fichero,
- un `int` en el offset 12 con el tamaño del bloque,
- un `int` en el offset 16 con el tamaño de la cabecera del bloque,
- un `int` en el offset 20 con el tamaño de la cabecera del registro,
- un `int` en el offset 24 con el número de atributos del registro,
- una secuencia de `int` a partir del offset 28 con el tipo de cada campo,
- una secuencia de `int` inmediatamente a continuación con la longitud de cada campo.

El campo almacenado en el offset 8 representa el número total de registros insertados en el fichero, incluidos los que hayan sido borrados lógicamente. Ese valor se usa tanto como identificador del próximo registro a insertar como límite superior de la lectura secuencial.

En este diseño se asume `sizeof(int) = 4`, de forma que todos los offsets anteriores se calculan en múltiplos de cuatro bytes.

3.2.2. Contenido de `blockInMemory`

La secuencia de bytes almacenada en `blockInMemory` contiene:

- la cabecera del bloque, formada por un único `int` con el identificador del bloque,
- una secuencia de registros de longitud fija.

Cada registro almacenado dentro del bloque contiene:

- un `int` con el identificador del registro dentro del fichero,
- un `int` con el estado del registro (`ACTIVO` o `BORRADO`),
- una secuencia de bytes con los valores de los campos del registro.

3.3. Operaciones públicas

- `fileASF()`
  Constructor por defecto. Inicializa la estructura sin asociarla todavía a ningún fichero. Los centinelas se inicializan a `false`, `loadedBlockId` se inicializa a `-1` y la posición de lectura secuencial se sitúa al principio.

- `void create(`

  `const std::string& filename,`

  `int blockSize,`

  `int blockHeaderSize,`

  `int recordHeaderSize,`

  `const std::vector<campType>& types,`

  `const std::vector<int>& lengths)`

  Crea un nuevo fichero ASF. Inicializa la cabecera del fichero en memoria, abre el fichero físico en modo creación y escribe la cabecera en disco.

- `void open(const std::string& filename)`
  Abre un fichero ASF ya existente. Lee la cabecera desde disco y la carga en `fileHeader`.

- `void close()`
  Cierra el fichero ASF. Antes de cerrar, vuelca a disco el bloque en memoria si ha sido modificado y vuelca también la cabecera del fichero si ha sido modificada.

- `void insertRecord(const fileRecord& record)`
  Inserta un nuevo registro al final del fichero.

  La precondición de esta operación es que la estructura del registro recibido coincida con la estructura definida por la cabecera del fichero.

  El nuevo registro recibe como identificador el valor actual de `recordCount`. A partir de ese identificador se calcula el bloque destino y la posición del registro dentro del bloque.

  Si el registro debe colocarse en el primer hueco de un bloque nuevo, se crea un nuevo bloque en memoria. Si antes había un bloque modificado pendiente de escritura, se escribe primero en disco. Si el bloque destino ya existía, se carga desde disco si no era el bloque actualmente cargado en memoria.

  Después se escriben en la posición correspondiente la cabecera del registro almacenado en el bloque (identificador y estado) y los datos del registro. Finalmente se actualizan en la cabecera del fichero el número total de registros insertados y, si procede, el número de bloques.

- `bool readNextRecord(fileRecord& record)`
  Lee el siguiente registro activo del fichero siguiendo el orden secuencial físico.

  La operación usa `currentBlockId` y `currentRecordIndex` para saber cuál es la posición actual de lectura. Para cada posición candidata se calcula el índice global del registro teórico dentro del fichero. Si ese índice es mayor o igual que el número total de registros insertados, la lectura termina y la operación devuelve `false`. Esto evita confundir huecos no inicializados del último bloque con registros realmente borrados.

  Si la posición examinada contiene un registro activo, la operación construye un objeto `fileRecord` con su estructura y sus datos, lo devuelve a través del parámetro de salida y devuelve `true`. Si el registro está marcado como borrado, continúa con el siguiente.

- `bool findRecord(`

  `int recordId,`

  `fileRecord& record)`

  Busca un registro activo por su identificador.

  Como los identificadores se asignan secuencialmente en el momento de inserción, el bloque que contiene un registro de identificador `recordId` se obtiene directamente a partir del número de registros por bloque, y lo mismo ocurre con la posición del registro dentro del bloque. Por tanto, esta operación no necesita recorrer secuencialmente todos los bloques.

  Si el registro existe y está activo, se devuelve en el parámetro `record` y la operación devuelve `true`. En caso contrario devuelve `false`.

- `bool deleteRecord(int recordId)`
  Marca como borrado el registro activo cuyo identificador se recibe como parámetro.

  La operación calcula directamente el bloque y la posición del registro dentro del bloque. Si el registro existe y su estado es `ACTIVO`, cambia ese estado a `BORRADO`, marca el bloque como modificado y devuelve `true`. Si no existe o ya estaba borrado, devuelve `false`.

- `bool modifyRecord(`

  `int recordId,`

  `const fileRecord& record)`

  Sustituye los datos de un registro activo por los datos contenidos en el objeto `record` recibido como parámetro.

  La operación calcula directamente el bloque y la posición del registro dentro del bloque. Si el registro existe y está activo, copia sus nuevos datos sobre la zona de datos del registro almacenado en el bloque, marca el bloque como modificado y devuelve `true`. Si no existe o está borrado, devuelve `false`.

- `void resetRead()`
  Reinicia la lectura secuencial. Sitúa `currentBlockId` y `currentRecordIndex` al principio del fichero para que una operación posterior de lectura secuencial vuelva a comenzar desde el primer registro.

3.4. Operaciones privadas

- `int getHeaderSize()`
  Lee de `fileHeader` el tamaño de la cabecera del fichero.

- `int getBlockCount()`
  Lee de `fileHeader` el número de bloques del fichero.

- `int getRecordCount()`
  Lee de `fileHeader` el número total de registros insertados en el fichero.

- `int getBlockSize()`
  Lee de `fileHeader` el tamaño de cada bloque.

- `int getBlockHeaderSize()`
  Lee de `fileHeader` el tamaño de la cabecera de bloque.

- `int getRecordHeaderSize()`
  Lee de `fileHeader` el tamaño de la cabecera de registro.

- `int getAttributeCount()`
  Lee de `fileHeader` el número de atributos del registro.

- `int getRecordDataSize()`
  Calcula la longitud total de la zona de datos de un registro sumando las longitudes de todos sus campos.

- `int getRecordSize()`
  Calcula el tamaño total de un registro almacenado dentro del bloque, es decir, tamaño de cabecera de registro más tamaño de datos.

- `int getRecordsPerBlock()`
  Calcula cuántos registros caben en un bloque.

- `std::vector<char>buildRecordStructure()`
  Construye la secuencia de bytes que representa la estructura del registro a partir de la información almacenada en `fileHeader`.

- `void readBlockFromDisk(int blockId)`
  Lee desde disco el bloque cuyo identificador es `blockId` y lo carga en `blockInMemory`. Si el bloque actualmente cargado en memoria ha sido modificado, la operación escribe primero ese bloque en disco. A continuación calcula el desplazamiento del bloque solicitado dentro del fichero, posiciona el puntero de lectura y lee el bloque completo. Finalmente actualiza `loadedBlockId` y desactiva el centinela de bloque modificado.

- `void writeBlockToDisk()`
  Escribe en disco el contenido actual de `blockInMemory`. El desplazamiento se calcula a partir de `loadedBlockId`. Después de la escritura, el centinela `hasBlockChanged` pasa a `false`.

- `void readFileHeaderFromDisk()`
  Lee la cabecera del fichero desde disco. Primero lee el entero inicial con el tamaño de la cabecera, redimensiona `fileHeader` a ese tamaño y después lee el resto del contenido. Finalmente desactiva el centinela `hasHeaderChanged`.

- `void writeFileHeaderToDisk()`
  Escribe en disco el contenido actual de `fileHeader` a partir del comienzo del fichero. Después de la escritura, el centinela `hasHeaderChanged` pasa a `false`.

- `int readInt(`

  ```
  const std::vector<char>& buffer,
  int offset)
  ```

  Lee cuatro bytes del buffer indicado a partir del desplazamiento `offset` y los interpreta como un valor entero.

- `void writeInt(`

  `std::vector<char>& buffer,`

  `int offset,`

  `int value)`

  Escribe el entero recibido en cuatro bytes consecutivos del buffer indicado, comenzando en el desplazamiento `offset`.

4. Análisis de operaciones

4.1. Operaciones que realizan lectura o escritura en disco

- `create`
  Realiza escritura en disco, ya que escribe la cabecera inicial del fichero. Modifica `diskFile`, que pasa a quedar abierto y asociado al fichero creado, y modifica `fileHeader`, que queda inicializado en memoria con la configuración del fichero y persistido en disco.

- `open`
  Realiza lectura desde disco, ya que carga la cabecera del fichero. Modifica `diskFile`, que pasa a quedar abierto y asociado al fichero existente, y modifica `fileHeader`, que queda cargado con la información leída desde disco. Además, tras la lectura, `hasHeaderChanged` queda a `false`.

- `close`
  Puede realizar escritura en disco. Si `hasBlockChanged` vale `true`, escribe `blockInMemory` en disco mediante `writeBlockToDisk()`. Si `hasHeaderChanged` vale `true`, escribe `fileHeader` en disco mediante `writeFileHeaderToDisk()`. Después de dichas escrituras, los centinelas correspondientes quedan a `false`. Finalmente modifica `diskFile`, que queda cerrado.

- `insertRecord`
  Puede realizar lectura o escritura en disco, directa o indirectamente.

  Si el registro debe insertarse en un bloque ya existente y dicho bloque no está cargado en memoria, la operación necesita leerlo desde disco mediante `readBlockFromDisk()`. Esa lectura puede provocar previamente una escritura en disco del bloque actualmente cargado si `hasBlockChanged` vale `true`.

  Si el registro debe insertarse en el primer hueco de un bloque nuevo, la operación puede provocar una escritura del bloque anterior si estaba modificado antes de inicializar el nuevo bloque en memoria.

  Modifica `blockInMemory`, que queda con el nuevo registro insertado; puede modificar `loadedBlockId`, que pasa a identificar el bloque destino; modifica `fileHeader`, incrementando el número total de registros insertados y, si procede, el número de bloques; y activa `hasBlockChanged` y `hasHeaderChanged`.

- `readNextRecord`
  Puede realizar lectura desde disco cuando necesita cargar un bloque distinto del que está actualmente en memoria.

Dicha lectura se realiza invocando `readBlockFromDisk` con `currentBlockId` como argumento y, por tanto, puede provocar previamente una escritura en disco del bloque actualmente cargado si `hasBlockChanged` vale `true`.

Modifica `blockInMemory`, que queda con el bloque leído; modifica `loadedBlockId`, que pasa a identificar ese bloque; y modifica `currentBlockId` y `currentRecordIndex`, que avanzan la posición de lectura secuencial.

- `findRecord`
  Puede realizar lectura desde disco cuando el bloque que contiene el registro buscado no está cargado en memoria.

  Dicha lectura se realiza invocando `readBlockFromDisk` con `blockId` como argumento y, por tanto, puede provocar previamente una escritura en disco del bloque actualmente cargado si `hasBlockChanged` vale `true`.

  Modifica `blockInMemory`, que queda con el bloque leído, y modifica `loadedBlockId`, que pasa a identificar ese bloque.

- `deleteRecord`
  Puede realizar lectura desde disco cuando necesita cargar el bloque que contiene el registro.

  Dicha lectura se realiza invocando `readBlockFromDisk` con `blockId` como argumento y, por tanto, puede provocar previamente una escritura en disco del bloque actualmente cargado si `hasBlockChanged` vale `true`.

  Si el registro existe y está activo, modifica `blockInMemory`, cambiando el estado del registro a `BORRADO`, y activa `hasBlockChanged`. La escritura efectiva del bloque puede quedar diferida hasta una operación posterior.

- `modifyRecord`
  Puede realizar lectura desde disco cuando necesita cargar el bloque que contiene el registro.

  Dicha lectura se realiza invocando `readBlockFromDisk` con `blockId` como argumento y, por tanto, puede provocar previamente una escritura en disco del bloque actualmente cargado si `hasBlockChanged` vale `true`.

  Si el registro existe y está activo, modifica `blockInMemory`, sobrescribiendo la zona de datos del registro, y activa `hasBlockChanged`. La escritura efectiva del bloque puede quedar diferida hasta una operación posterior.

- `readBlockFromDisk`
  Realiza lectura desde disco del bloque solicitado. Si `hasBlockChanged` vale `true`, antes de leer realiza una escritura en disco del bloque actualmente cargado mediante `writeBlockToDisk()`.

  Modifica `blockInMemory`, que queda con el contenido del bloque leído; modifica `loadedBlockId`, que pasa a identificar el bloque recién cargado; y deja `hasBlockChanged` a `false`.

- `writeBlockToDisk`
  Realiza escritura en disco del contenido actual de `blockInMemory`. Después de la escritura, `hasBlockChanged` queda a `false`.

- `readFileHeaderFromDisk`
  Realiza lectura desde disco de la cabecera del fichero. Modifica `fileHeader`, que queda cargado con el contenido leído, y deja `hasHeaderChanged` a `false`.

- `writeFileHeaderToDisk`
  Realiza escritura en disco del contenido actual de `fileHeader`. Después de la escritura, `hasHeaderChanged` queda a `false`.

Las operaciones del TDA `fileRecord` no realizan lectura ni escritura en disco. Tampoco realizan acceso a disco `fileASF()`, `resetRead()`, `getHeaderSize()`, `getBlockCount()`, `getRecordCount()`, `getBlockSize()`, `getBlockHeaderSize()`, `getRecordHeaderSize()`, `getAttributeCount()`, `getRecordDataSize()`, `getRecordSize()`, `getRecordsPerBlock()` y `buildRecordStructure()`.

4.2. Operaciones que escriben `fileHeader` en disco

Las operaciones que escriben la cabecera del fichero en disco son:

- `create`, al crear el fichero e inicializar su cabecera.
- `close`, si `hasHeaderChanged` vale `true`.
- `writeFileHeaderToDisk`, siempre que es invocada.

4.3. Operaciones que abren o cierran `diskFile`

- `create` abre el fichero.
- `open` abre el fichero.
- `close` cierra el fichero.

4.4. Operaciones que desplazan la posición actual del fichero

Para describir con precisión los desplazamientos de la posición del fichero, se expresan siempre mediante un offset absoluto respecto del origen `ios::beg`. De este modo, la posición se fija explícitamente y no se depende de la posición previa del fichero.

Sea:

$$\begin{aligned} \texttt{headerSize} &= \texttt{getHeaderSize()} \\ \texttt{blockSize} &= \texttt{getBlockSize()} \\ \texttt{rPerBlock} &= \texttt{getRecordsPerBlock()} \end{aligned}$$

- `create`
  Sitúa la posición de escritura al comienzo del fichero para escribir la cabecera inicial. Equivale a escribir desde el offset 0 respecto de `ios::beg`.

- `open`
  Desplaza la posición indirectamente mediante `readFileHeaderFromDisk()`, que sitúa la lectura al comienzo del fichero, es decir, en el offset 0 respecto de `ios::beg`.

- `close`
  Puede desplazar la posición indirectamente:
  - si `hasBlockChanged` vale `true`, mediante `writeBlockToDisk()`,
  - si `hasHeaderChanged` vale `true`, mediante `writeFileHeaderToDisk()`.
- `readFileHeaderFromDisk`
  Desplaza la posición al comienzo del fichero mediante `seekg(0, ios::beg)`. Primero lee 4 bytes para obtener el tamaño de la cabecera y, a continuación, lee el resto hasta completar `headerSize` bytes en total.
- `writeFileHeaderToDisk`
  Desplaza la posición al comienzo del fichero mediante `seekp(0, ios::beg)` y escribe `headerSize` bytes.
- `readBlockFromDisk`
  Puede provocar dos desplazamientos.

  Si `hasBlockChanged` vale `true`, primero desplaza la posición de escritura hasta el comienzo del bloque actualmente cargado mediante `writeBlockToDisk()`, con offset

  $$\texttt{headerSize} + \texttt{loadedBlockId} \cdot \texttt{blockSize}$$

  respecto de `ios::beg`.

  Después desplaza la posición de lectura hasta el comienzo del bloque solicitado, con offset

  $$\texttt{headerSize} + \texttt{blockId} \cdot \texttt{blockSize}$$

  respecto de `ios::beg`, y desde ahí lee `blockSize` bytes.
- `writeBlockToDisk`
  Desplaza la posición de escritura hasta el comienzo del bloque actualmente cargado en memoria, con offset

  $$\texttt{headerSize} + \texttt{loadedBlockId} \cdot \texttt{blockSize}$$

  respecto de `ios::beg`, y desde ahí escribe `blockSize` bytes.
- `insertRecord`
  Desplaza la posición indirectamente cuando necesita cargar o volcar bloques.

  Si `targetBlockId` es el identificador del bloque destino del nuevo registro, entonces

  $$\texttt{targetBlockId} = \texttt{getRecordCount()}/\texttt{rPerBlock}$$

  Si el bloque destino no está cargado en memoria, la operación desplaza la posición mediante `readBlockFromDisk(targetBlockId)`. Si antes debe volcarse el bloque actualmente cargado, el desplazamiento se realiza mediante `writeBlockToDisk()`.

  Si el bloque destino ya está en memoria y no hay que volcar ningún bloque previo, no necesita desplazar la posición del fichero en ese momento.

- `readNextRecord`
  Desplaza la posición indirectamente cuando necesita cargar un bloque distinto del actualmente cargado en memoria.

  En ese caso invoca `readBlockFromDisk(currentBlockId)`, por lo que el offset utilizado es

  $$\texttt{headerSize} + \texttt{currentBlockId} \cdot \texttt{blockSize}$$

  respecto de `ios::beg`, además de la posible escritura previa del bloque cargado si estaba modificado.

- `findRecord`
  Desplaza la posición indirectamente mediante `readBlockFromDisk(blockId)` cuando el bloque buscado no está en memoria, donde

  $$\texttt{blockId} = \texttt{recordId}/\texttt{rPerBlock}$$

  El offset de lectura es

  $$\texttt{headerSize} + \texttt{blockId} \cdot \texttt{blockSize}$$

  respecto de `ios::beg`, además de la posible escritura previa del bloque cargado si estaba modificado.

- `deleteRecord`
  Desplaza la posición indirectamente mediante `readBlockFromDisk(blockId)` cuando el bloque que contiene el registro no está en memoria, donde

  $$\texttt{blockId} = \texttt{recordId}/\texttt{rPerBlock}$$

  El offset de lectura es

  $$\texttt{headerSize} + \texttt{blockId} \cdot \texttt{blockSize}$$

  respecto de `ios::beg`, además de la posible escritura previa del bloque cargado si estaba modificado.

- `modifyRecord`
  Desplaza la posición indirectamente mediante `readBlockFromDisk(blockId)` cuando el bloque que contiene el registro no está en memoria, donde

  $$\texttt{blockId} = \texttt{recordId}/\texttt{rPerBlock}$$

  El offset de lectura es

  $$\texttt{headerSize} + \texttt{blockId} \cdot \texttt{blockSize}$$

  respecto de `ios::beg`, además de la posible escritura previa del bloque cargado si estaba modificado.

Las operaciones del TDA `fileRecord` no desplazan la posición actual del fichero. Tampoco lo hacen por sí mismas `fileASF()`, `resetRead()`, `getHeaderSize()`, `getBlockCount()`, `getRecordCount()`, `getBlockSize()`, `getBlockHeaderSize()`, `getRecordHeaderSize()`, `getAttributeCount()`, `getRecordDataSize()`, `getRecordSize()`, `getRecordsPerBlock()` y `buildRecordStructure()`.